



Konrad Fugiel (134909) Oliwier Hędrzak (134913)
Filip Lisowski (134942)

Informatyka

*Implementacja algorytmów sztucznej inteligencji na przykładzie
gier Nim i Reversi – dokumentacja projektowa*

Praca projektowa

Praca wykonana pod kierunkiem
mgr inż. Marcin Mrukowicz

Spis treści

1. Cel i zakres projektu	7
1.1. Cel projektu	7
1.2. Zakres projektu	7
1.3. Zrealizowane moduły	7
2. Zasady gier i założenia projektowe	8
2.1. Zasady gry Reversi	8
2.2. Zasady gry Nim	8
2.3. Tryby rozgrywki	8
2.4. Założenia dotyczące agentów AI	8
3. Struktura i architektura aplikacji	9
3.1. Ogólna struktura katalogów	9
3.2. Podział na warstwy aplikacji	9
3.3. Modele danych	9
3.4. Silniki reguł gry	9
3.5. Warstwa interfejsu graficznego	9
4. Algorytmy sztucznej inteligencji i strategię decyzyjne	10
4.1. Agent decyzyjny gry Reversi	10
4.1.1. Charakterystyka problemu decyzyjnego w Reversi	10
4.1.2. Algorytm Minimax z przycinaniem Alpha-Beta	10
4.1.3. Funkcja heurystyczna oceny planszy	13
4.1.4. Implementacja agenta Reversi w projekcie	14
4.1.5. Tryb AI vs AI i wpływ głębokości przeszukiwania	14
4.2. Strategia optymalna w grze Nim	14
4.2.1. Charakterystyka gry Nim jako problemu decyzyjnego	14
4.2.2. Nim-sum jako podstawa decyzji	15
4.2.3. Wyznaczanie ruchu optymalnego	15
4.2.4. Implementacja agenta Nim w projekcie	16
4.2.5. Tryb AI vs AI i deterministyczny przebieg rozgrywki	17
5. Implementacja interfejsu użytkownika	18
5.1. Interfejs gry Reversi	18
5.2. Interfejs gry Nim	18
5.3. Obsługa trybu Człowiek vs AI	18
5.4. Obsługa trybu AI vs AI	18
5.5. Prezentacja historii ruchów i analizy AI	18
6. Przykładowe przebiegi rozgrywek	19
6.1. Przykład działania Reversi w trybie Człowiek vs AI	19

6.2.	Przykład działania Reversi w trybie AI vs AI	19
6.3.	Przykład działania Nim w trybie Człowiek vs AI	19
6.4.	Przykład działania Nim w trybie AI vs AI	19
6.5.	Analiza wybranych decyzji agentów	19
7.	Napotkane wyzwania i sposoby ich rozwiązania.....	20
7.1.	Separacja logiki gry od interfejsu.....	20
7.2.	Problem wydajności w grze Reversi.....	20
7.3.	Dobór funkcji heurystycznej	20
7.4.	Obsługa trybu AI vs AI	20
7.5.	Problemy z interfejsem i prezentacją stanu gry.....	20
8.	Instrukcja uruchomienia projektu	21
8.1.	Wymagania środowiskowe	21
8.2.	Struktura projektu.....	21
8.3.	Uruchomienie gry Nim.....	22
8.4.	Uruchomienie gry Reversi	22
9.	Podsumowanie i wnioski.....	23
9.1.	Zrealizowane funkcjonalności	23
9.2.	Ocena działania agentów AI.....	23
9.3.	Możliwe kierunki rozwoju	23
	Bibliografia	24
	Spis rysunków	25
	Oświadczenie studenta o samodzielności pracy	26

1. Cel i zakres projektu

1.1. Cel projektu

1.2. Zakres projektu

1.3. Zrealizowane moduły

2. Zasady gier i założenia projektowe

2.1. Zasady gry Reversi

2.2. Zasady gry Nim

2.3. Tryby rozgrywki

2.4. Założenia dotyczące agentów AI

3. Struktura i architektura aplikacji

3.1. Ogólna struktura katalogów

3.2. Podział na warstwy aplikacji

3.3. Modele danych

3.4. Silniki reguł gry

3.5. Warstwa interfejsu graficznego

4. Algorytmy sztucznej inteligencji i strategię decyzyjne

Rozdział opisuje mechanizmy decyzyjne wykorzystane w obu grach. W Reversi zastosowano algorytm Minimax z przycinaniem Alpha-Beta oraz heurystyczną ocenę planszy. W grze Nim wykorzystano strategię optymalną opartą na operacji XOR i wartości Nim-sum.

Omówiono zarówno podstawy teoretyczne algorytmów, jak i sposób ich zastosowania w projekcie. Uwzględniono również tryby AI vs AI, które pozwalają obserwować działanie agentów bez udziału gracza.

4.1. Agent decyzyjny gry Reversi

4.1.1. Charakterystyka problemu decyzyjnego w Reversi

Reversi jest deterministyczną grą dwuosobową o pełnej informacji. Oznacza to, że obaj gracze znają aktualny stan planszy, rozmieszczenie pionków oraz dostępne ruchy. W projekcie wykorzystano klasyczną planszę 8×8 .

Z punktu widzenia AI stan gry obejmuje układ pionków oraz informację, który gracz wykonuje ruch. Agent musi wybrać jeden z legalnych ruchów, ale nie może kierować się wyłącznie chwilową liczbą zdobytych pionków. W Reversi pozornie korzystny ruch może w kolejnych turach prowadzić do utraty ważnych pól, np. rogów planszy.

Z tego powodu zastosowano algorytm Minimax, który analizuje możliwe odpowiedzi przeciwnika. Pełne przeszukanie drzewa gry jest zbyt kosztowne, dlatego agent analizuje stany tylko do określonej głębokości, a następnie ocenia planszę za pomocą funkcji heurystycznej.

4.1.2. Algorytm Minimax z przycinaniem Alpha-Beta

Podstawowym algorytmem decyzyjnym w grze Reversi jest Minimax. Algorytm zakłada, że jedna strona maksymalizuje wynik, a przeciwnik wybiera ruchy minimalizujące ten wynik. W projekcie graczem maksymalizującym jest zawsze agent, dla którego aktualnie liczona jest decyzja.

Drzewo gry nie jest analizowane do końca rozgrywki, ponieważ liczba możliwych kontynuacji szybko rośnie. Po osiągnięciu zadanej głębokości algorytm korzysta z funkcji heurystycznej, która szacuje jakość aktualnej planszy.

W celu ograniczenia liczby analizowanych stanów zastosowano przycinanie Alpha-Beta. Mechanizm wykorzystuje dwie wartości:

- α – najlepszy dotychczas znaleziony wynik gracza maksymalizującego,
- β – najlepszy dotychczas znaleziony wynik gracza minimalizującego.

Jeżeli spełniony zostanie warunek:

$$\beta \leq \alpha,$$

dalsze analizowanie danej gałęzi zostaje przerwane. Alpha-Beta nie zmienia wyniku Minimaxu, ale pozwala pominąć obliczenia, które nie wpłynęłyby na końcową decyzję.

Agent zapisuje również statystyki działania algorytmu: liczbę ocenionych stanów, liczbę odcięć Alpha-Beta oraz czas potrzebny na wybór ruchu. Dane te są prezentowane w panelu analizy AI i historii ruchów.

Listing 4.1. Najważniejszy fragment metody minimax() z przycinaniem Alpha-Beta.

```
if current_player == ai_player:
    best_score = -inf

    for move in self.order_moves(valid_moves):
        next_state = self.engine.apply_move(
            GameState(board=board, current_player=current_player),
            move
        )

        score = self.minimax(
            next_state.board,
            depth - 1,
            alpha,
            beta,
            next_state.current_player,
            ai_player
        )

        best_score = max(best_score, score)
        alpha = max(alpha, best_score)

        if beta <= alpha:
            self.stats.alpha_beta_cutoffs += 1
            break

    return int(best_score)

best_score = inf

for move in self.order_moves(valid_moves):
    next_state = self.engine.apply_move(
        GameState(board=board, current_player=current_player),
        move
    )

    score = self.minimax(
        next_state.board,
        depth - 1,
        alpha,
        beta,
        next_state.current_player,
        ai_player
    )

    best_score = min(best_score, score)
    beta = min(beta, best_score)

    if beta <= alpha:
        self.stats.alpha_beta_cutoffs += 1
        break

return int(best_score)
```

Listing 4.1 przedstawia rdzeń metody `minimax()`, czyli fazę maksymalizacji, fazę minimalizacji oraz warunek odcięcia Alpha-Beta. Pominięto fragmenty techniczne, takie jak przygotowanie listy ruchów, obsługa braku ruchu oraz warunek stopu.

W fazie MAX algorytm wybiera najwyższą wartość oceny i aktualizuje α . W fazie MIN wybierana jest najniższa wartość i aktualizowane jest β . Gdy zachodzi warunek $\beta \leq \alpha$, dalsza analiza gałęzi zostaje przerwana, a licznik odcięć zostaje zwiększony.

Przed wejściem do przedstawionego fragmentu metoda sprawdza, czy osiągnięto maksymalną głębokość albo stan końcowy. Jeśli tak, plansza jest oceniana przez metodę `evaluate_terminal_or_heuristic()`.

Listing 4.2. Fragment wyboru najlepszego ruchu przez agenta Reversi.

```
best_move = None
best_score = -inf

for move in self.order_moves(valid_moves):
    next_state = self.engine.apply_move(
        GameState(board=board, current_player=current_player),
        move
    )

    score = self.minimax(
        board=next_state.board,
        depth=depth - 1,
        alpha=-inf,
        beta=inf,
        current_player=next_state.current_player,
        ai_player=ai_player
    )

    if score > best_score:
        best_score = score
        best_move = move
```

Listing 4.2 pokazuje uproszczony wybór najlepszego ruchu. Każdy legalny ruch jest symulowany na tymczasowej planszy, a następnie oceniany przez `minimax()`. Ruch z najwyższą oceną zostaje zapamiętany jako decyzja agenta.

W pełnej implementacji metoda `choose_best_move()` dodatkowo mierzy czas analizy, resetuje statystyki, obsługuje brak legalnych ruchów i zwraca wynik w strukturze `MinimaxResult`.

W trybie Człowiek vs AI głębokość przeszukiwania zależy od wybranego poziomu trudności. W trybie AI vs AI każdy agent może mieć osobną głębokość. AI 1 gra czarnymi pionkami, a AI 2 białymi.

Listing 4.3. Wywołanie agenta AI w trybie AI vs AI.

```
result = self.ai.choose_best_move(
    board=self.game_state.board,
    current_player=current_player,
    ai_player=current_player,
    depth=depth
)
```

Listing 4.3 pokazuje użycie agenta w trybie AI vs AI. Aktualny gracz jest jednocześnie graczem, dla którego liczona jest najlepsza decyzja. Dzięki temu ta sama klasa AI obsługuje obu agentów.

4.1.3. Funkcja heurystyczna oceny planszy

Minimax w Reversi nie przeszukuje całego drzewa gry do końca rozgrywki. Po osiągnięciu ustalonej głębokości wykorzystywana jest funkcja heurystyczna, która przypisuje planszy wartość liczbową określającą, czy dany stan jest korzystny dla agenta.

Ocena planszy opiera się na kilku elementach strategicznych. Najważniejsze są rogi, ponieważ pionków w rogach nie można odwrócić. Uwzględniana jest także mobilność, czyli liczba legalnych ruchów gracza i przeciwnika.

Funkcja bierze również pod uwagę wartość pozycyjną pól, różnicę pionków oraz karę za zajmowanie pól ryzykownych przy pustych rogach. Dzięki temu agent nie ocenia pozycji wyłącznie na podstawie chwilowej liczby pionków.

Ogólną postać funkcji heurystycznej można przedstawić jako sumę ważoną:

$$H(s) = w_c \cdot C(s) + w_m \cdot M(s) + w_p \cdot P(s) + w_d \cdot D(s) - w_r \cdot R(s),$$

gdzie:

- $C(s)$ oznacza ocenę zajętych rogów,
- $M(s)$ oznacza ocenę mobilności,
- $P(s)$ oznacza ocenę pozycyjną pól planszy,
- $D(s)$ oznacza różnicę liczby pionków,
- $R(s)$ oznacza karę za pola ryzykowne,
- w_c, w_m, w_p, w_d, w_r oznaczają wagi składników.

Listing 4.4. Fragment funkcji heurystycznej oceny planszy Reversi.

```
def evaluate_board(self, board: list[list[int]], ai_player: int) -> int:
    components = self.evaluate_components(board, ai_player)

    return (
        self.CORNER_WEIGHT * components["corner_score"]
        + self.MOBILITY_WEIGHT * components["mobility_score"]
        + self.POSITIONAL_WEIGHT * components["positional_score"]
        + self.DISC_DIFFERENCE_WEIGHT * components["disc_difference"]
        - self.RISKY_SQUARE_WEIGHT * components["risky_square_penalty"]
    )
```

Listing 4.4 przedstawia sposób obliczania oceny heurystycznej. Metoda `evaluate_board()` pobiera wartości cząstkowe i łączy je w jeden wynik liczbowy. Wynik dodatni oznacza pozycję korzystną dla agenta, a ujemny przewagę przeciwnika.

Największy wpływ na ocenę mają rogi planszy i mobilność. Uwzględniana jest także macierz wag pól, różnica pionków oraz kara za pola ryzykowne przy pustych rogach. Ocena heurystyczna jest oddzielona od Minimaxu, dzięki czemu można łatwo zmieniać wagi lub dodawać nowe składniki.

4.1.4. Implementacja agenta Reversi w projekcie

Agent Reversi został zaimplementowany w klasie `ReversiAI`. Korzysta ona z silnika zasad gry `ReversiEngine` oraz modułu oceny heurystycznej `ReversiEvaluator`. Dzięki temu wybór ruchu jest oddzielony od logiki zasad gry i interfejsu graficznego.

Głównym punktem wejścia jest metoda `choose_best_move()`. Otrzymuje ona aktualną planszę, gracza wykonującego ruch, gracza analizowanego przez AI oraz głębokość przeszukiwania. Następnie sprawdza legalne ruchy, symuluje je i wybiera najlepszy wynik zwrócony przez `minimax()`.

Wynik decyzji zapisywany jest w strukturze `MinimaxResult`. Zawiera ona wybrany ruch, ocenę punktową oraz statystyki, takie jak liczba ocenionych stanów, liczba odcięć Alpha-Beta i czas analizy.

W implementacji zastosowano także metodę `order_moves()`, która analizuje ruchy prowadzące do zajęcia rogów jako pierwsze. Poprawia to skuteczność przycinania Alpha-Beta. Agent obsługuje również sytuację braku legalnego ruchu, przekazując wtedy turę przeciwnikowi.

Ta sama klasa `ReversiAI` jest używana w trybie Człowiek vs AI oraz AI vs AI. Różnią się jedynie parametry przekazywane do metody `choose_best_move()`.

4.1.5. Tryb AI vs AI i wpływ głębokości przeszukiwania

W trybie AI vs AI obie strony są sterowane przez klasę `ReversiAI`. AI 1 gra czarnymi pionkami, a AI 2 białymi. Pozwala to obserwować rozgrywkę bez udziału gracza i porównywać decyzje agentów.

Każdy agent może mieć osobną głębokość przeszukiwania. Większa głębokość pozwala analizować dalsze konsekwencje ruchu, ale zwiększa liczbę sprawdzanych stanów i czas obliczeń.

Podczas ruchu aktualny gracz jest przekazywany do algorytmu jako gracz maksymalizujący. Dzięki temu AI 1 ocenia planszę z perspektywy czarnych pionków, a AI 2 z perspektywy białych.

Tryb AI vs AI pełni funkcję demonstracyjną. Pokazuje działanie Minimaxu z Alpha-Beta, wpływ głębokości przeszukiwania oraz statystyki decyzji, takie jak liczba ocenionych stanów, liczba odcięć i czas analizy.

4.2. Strategia optymalna w grze Nim

4.2.1. Charakterystyka gry Nim jako problemu decyzyjnego

Nim jest deterministyczną grą dwuosobową o pełnej informacji. Stan gry jest opisany przez listę stert, z których każda zawiera określoną liczbę żetonów. W pojedynczym ruchu gracz wybiera jedną stertę i zabiera z niej co najmniej jeden żeton. W zastosowanym wariantcie wygrywa gracz, który zabierze ostatni żeton.

W przeciwieństwie do Reversi, gra Nim nie wymaga przeszukiwania drzewa gry za pomocą algorytmu Minimax. Dla tej gry istnieje strategia optymalna oparta na operacji bitowej XOR. Dzięki temu agent może bezpośrednio obliczyć, czy aktualny stan jest wygrywający, oraz wskazać ruch prowadzący do pozycji niekorzystnej dla przeciwnika.

Problem decyzyjny w Nim polega więc na analizie wartości stert i wyznaczeniu takiego ruchu, który po wykonaniu pozostawi przeciwnikowi pozycję przegrywającą przy optymalnej grze.

4.2.2. Nim-sum jako podstawa decyzji

Podstawą strategii agenta w grze Nim jest wartość Nim-sum. Jest ona obliczana jako wynik operacji XOR wykonanej na liczbach żetonów we wszystkich stertach. W projekcie wartość ta pełni rolę sumy kontrolnej aktualnej pozycji.

Ogólny zapis Nim-sum można przedstawić następująco:

$$S = a_1 \oplus a_2 \oplus \dots \oplus a_n,$$

gdzie $a_1, a_2, \dots, a_n$ oznaczają liczby żetonów w kolejnych stertach, natomiast symbol $\oplus$ oznacza operację XOR.

Jeżeli wartość Nim-sum wynosi zero, aktualna pozycja jest przegrywająca przy optymalnej grze przeciwnika. Oznacza to, że gracz wykonujący ruch nie ma strategii gwarantującej zwycięstwo. Jeżeli Nim-sum jest różny od zera, pozycja jest wygrywająca i istnieje ruch, który sprowadza Nim-sum do wartości zero.

Listing 4.5. Obliczanie wartości Nim-sum dla aktualnych stert.

```
def calculate_nim_sum(self, piles: list[int]) -> int:
    nim_sum = 0

    for pile in piles:
        nim_sum ^= pile

    return nim_sum
```

Listing 4.5 przedstawia sposób obliczania wartości Nim-sum w projekcie. Algorytm przechodzi przez kolejne sterty i wykonuje operację XOR na ich wartościach. Otrzymany wynik określa typ pozycji: wygrywającą albo przegrywającą.

4.2.3. Wyznaczanie ruchu optymalnego

Jeżeli Nim-sum jest różny od zera, agent szuka sterty, którą można zmniejszyć tak, aby po ruchu całkowity Nim-sum wynosił zero. W tym celu dla każdej sterty obliczana jest wartość docelowa:

$$t_i = a_i \oplus S,$$

gdzie a_i oznacza aktualną wartość sterty, a S oznacza Nim-sum.

Jeżeli wartość t_i jest mniejsza od aktualnej liczby żetonów w stercie, możliwe jest wykonanie ruchu optymalnego. Agent zabiera wtedy:

$$a_i - t_i$$

żetonów z wybranej sterty. Po takim ruchu przeciwnik otrzymuje pozycję o wartości Nim-sum równej zero.

Jeżeli Nim-sum już na początku wynosi zero, agent nie posiada ruchu gwarantującego zwycięstwo. W takiej sytuacji wykonuje pierwszy dostępny poprawny ruch. W projekcie pełni to rolę ruchu awaryjnego.

Listing 4.6. Wyznaczanie optymalnego ruchu agenta Nim.

```
def find_optimal_move(self, piles: list[int]) -> Move:
    nim_sum = self.engine.calculate_nim_sum(piles)

    if nim_sum == 0:
        return self.find_fallback_move(piles)

    for index, pile in enumerate(piles):
        target = pile ^ nim_sum

        if target < pile:
            amount = pile - target
            return Move(index, amount)

    return self.find_fallback_move(piles)
```

Listing 4.6 pokazuje główną logikę wyboru ruchu. Agent najpierw oblicza Nim-sum. Jeżeli wynik wynosi zero, wybierany jest ruch awaryjny. W przeciwnym przypadku sprawdzane są kolejne sterty i wybierana jest taka, którą można zmniejszyć do wartości zapewniającej Nim-sum równy zero.

4.2.4. Implementacja agenta Nim w projekcie

Agent gry Nim został zaimplementowany w klasie `NimAI`. Klasa korzysta z silnika `NimEngine`, który odpowiada za obliczanie Nim-sum, walidację ruchów oraz wykonywanie ruchów na aktualnym stanie gry.

Główną metodą agenta jest `find_optimal_move()`. Metoda ta analizuje aktualne sterty i zwraca obiekt `Move`, zawierający indeks wybranej sterty oraz liczbę żetonów do zabrania. Jeżeli pozycja jest wygrywająca, agent wybiera ruch sprowadzający Nim-sum do zera. Jeżeli pozycja jest przegrywająca, wykonywany jest pierwszy poprawny ruch.

W projekcie dodano również metodę `explain_move()`, która przygotowuje tekstowe wyjaśnienie decyzji AI. Pokazuje ona stan przed ruchem, wartość Nim-sum, wybraną stertę, liczbę zabranych żetonów oraz stan po wykonaniu ruchu. Dzięki temu decyzja agenta jest czytelna dla użytkownika.

Silnik gry obsługuje także zmianę aktualnego gracza. W trybie Człowiek vs AI tura przechodzi pomiędzy graczami `human` i `ai`, natomiast w trybie AI vs AI pomiędzy `ai_1` i `ai_2`.

Listing 4.7. Fragment obsługi zmiany gracza w silniku gry Nim.

```
next_player = game_state.current_player

if not game_over:
    if game_state.current_player == "human":
        next_player = "ai"
    elif game_state.current_player == "ai":
        next_player = "human"
    elif game_state.current_player == "ai_1":
        next_player = "ai_2"
    elif game_state.current_player == "ai_2":
        next_player = "ai_1"

return GameState(
    piles=new_piles,
    current_player=next_player,
    game_over=game_over,
    winner=winner
)
```

Listing 4.7 przedstawia fragment logiki odpowiedzialnej za zmianę aktualnego gracza po wykonaniu ruchu. Dzięki temu ten sam silnik może obsługiwać zarówno tryb Człowiek vs AI, jak i AI vs AI.

4.2.5. Tryb AI vs AI i deterministyczny przebieg rozgrywki

W grze Nim zaimplementowano również tryb AI vs AI. W tym trybie obie strony korzystają z tej samej klasy `NimAI`, a ruchy wykonywane są automatycznie. Pierwszy agent oznaczony jest jako `ai_1`, natomiast drugi jako `ai_2`.

Rozgrywka AI vs AI w Nim ma charakter deterministyczny. Oznacza to, że dla tego samego początkowego układu stert agenci wykonają zawsze te same ruchy. Wynika to z faktu, że strategia nie korzysta z losowości, a sterty są sprawdzane w stałej kolejności.

Tryb AI vs AI pełni funkcję demonstracyjną. Pozwala pokazać działanie strategii opartej na Nim-sum oraz obserwować, w jaki sposób agent sprowadza przeciwnika do pozycji przegrywającej. W historii ruchów zapisywane są decyzje obu agentów, co ułatwia prześledzenie przebiegu partii.

W porównaniu z Reversi, gra Nim jest znacznie prostsza obliczeniowo. Agent nie musi analizować wielu przyszłych wariantów, ponieważ ruch optymalny można wyznaczyć bezpośrednio na podstawie aktualnego stanu stert.

5. Implementacja interfejsu użytkownika

5.1. Interfejs gry Reversi

5.2. Interfejs gry Nim

5.3. Obsługa trybu Człowiek vs AI

5.4. Obsługa trybu AI vs AI

5.5. Prezentacja historii ruchów i analizy AI

6. Przykładowe przebiegi rozgrywek

6.1. Przykład działania Reversi w trybie Człowiek vs AI

6.2. Przykład działania Reversi w trybie AI vs AI

6.3. Przykład działania Nim w trybie Człowiek vs AI

6.4. Przykład działania Nim w trybie AI vs AI

6.5. Analiza wybranych decyzji agentów

7. Napotkane wyzwania i sposoby ich rozwiązania

7.1. Separacja logiki gry od interfejsu

7.2. Problem wydajności w grze Reversi

7.3. Dobór funkcji heurystycznej

7.4. Obsługa trybu AI vs AI

7.5. Problemy z interfejsem i prezentacją stanu gry

8. Instrukcja uruchomienia projektu

Projekt składa się z dwóch niezależnych gier: Nim oraz Reversi. Obie gry zostały napisane w języku Python i korzystają z biblioteki `tkinter` do obsługi interfejsu graficznego.

8.1. Wymagania środowiskowe

Do uruchomienia projektu wymagane jest środowisko Python w wersji 3.12 lub nowszej. Interfejs graficzny wykorzystuje bibliotekę `tkinter`, która jest częścią standardowej dystrybucji Pythona.

Wymagania:

- Python 3.12 lub nowszy,
- terminal lub środowisko IDE, np. PyCharm.

8.2. Struktura projektu

Projekt został podzielony na dwa główne moduły: `nim_project` oraz `reversi_project`. Każdy z nich posiada podobną strukturę katalogów, co ułatwia analizę kodu i pokazuje spójny sposób organizacji projektu.

Listing 8.1. Uproszczona struktura projektu.

```
SI_project/  
|-- nim_project/  
|   |-- main.py  
|   |-- ai/  
|   |-- gui/  
|   |-- logic/  
|   |-- models/  
|  
|-- reversi_project/  
|   |-- main.py  
|   |-- ai/  
|   |-- gui/  
|   |-- logic/  
|   |-- models/  
|  
|-- .gitignore
```

Katalog `ai` zawiera logikę agentów decyzyjnych. Katalog `logic` odpowiada za zasady gry, poprawność ruchów i aktualizację stanu rozgrywki. W katalogu `models` znajdują się proste klasy danych, natomiast `gui` zawiera implementację interfejsu graficznego.

8.3. Uruchomienie gry Nim

Aby uruchomić grę Nim, należy przejść w terminalu do głównego katalogu projektu `SI_project`. Następnie należy wykonać polecenie:

```
python -m nim_project.main
```

Po uruchomieniu programu pojawia się ekran wyboru początkowego układu stert oraz trybu gry. Użytkownik może wybrać tryb Człowiek vs AI albo AI vs AI. W trybie Człowiek vs AI gracz sam wybiera żetony do zabrania, natomiast w trybie AI vs AI rozgrywka przebiega automatycznie.

8.4. Uruchomienie gry Reversi

Aby uruchomić grę Reversi, również należy znajdować się w katalogu głównym projektu `SI_project`. Następnie należy wykonać polecenie:

```
python -m reversi_project.main
```

Po uruchomieniu aplikacji wyświetlana jest plansza Reversi oraz panel boczny. Użytkownik może wybrać tryb Człowiek vs AI albo AI vs AI. W trybie Człowiek vs AI dostępny jest wybór poziomu trudności oraz podpowiedzi. W trybie AI vs AI można ustawić osobną głębokość przeszukiwania dla AI 1 i AI 2.

9. Podsumowanie i wnioski

9.1. Zrealizowane funkcjonalności

9.2. Ocena działania agentów AI

9.3. Możliwe kierunki rozwoju

Bibliografia

- [1] AlgoMonster. Nim game - analysis and solutions. <https://algo.monster/liteproblems/292>. Dostęp: 16.05.2026.
- [2] Archimedes Laboratory. Most wanted solutions: Nim game strategy. https://www.archimedes-lab.org/How_to_Solve/Win_at_Nim.html. Dostęp: 16.05.2026.
- [3] Archimedes Laboratory. Play nim game online. https://www.archimedes-lab.org/game_nim/play_nim_game.html. Dostęp: 16.05.2026.
- [4] GeeksforGeeks. Combinatorial game theory | set 2 (game of nim). <https://www.geeksforgeeks.org/dsa/combinatorial-game-theory-set-2-game-nim/>. Dostęp: 16.05.2026.
- [5] GeeksforGeeks. Introduction to combinatorial game theory. <https://www.geeksforgeeks.org/dsa/introduction-to-combinatorial-game-theory/>. Dostęp: 16.05.2026.
- [6] Marc Jethro. Nim-game: Implementation in python with gui and ai. <https://github.com/marcjethro/Nim-Game>. Dostęp: 16.05.2026.
- [7] OpenAI. Chatgpt – generative artificial intelligence assistant for gui development and refactoring. <https://chatgpt.com>. Wsparcie w projektowaniu układu i refaktoryzacji kodu GUI (PySide6); Dostęp: 16.05.2026.
- [8] Transum Software. Nim mathematical game of strategy (multi-pile version). <https://www.transum.org/Software/Nim/>. Dostęp: 16.05.2026.
- [9] Transum Software. The subtraction game (basic nim version). <https://www.transum.org/Software/Nim/Subtraction.asp>. Dostęp: 16.05.2026.
- [10] Wikipedia. Impartial game – the free encyclopedia. https://en.wikipedia.org/wiki/Impartial_game. Dostęp: 16.05.2026.
- [11] Wikipedia. Misère – the free encyclopedia. <https://en.wikipedia.org/wiki/Mis%C3%A8re>. Dostęp: 16.05.2026.
- [12] Wikipedia. Nim – the free encyclopedia. <https://en.wikipedia.org/wiki/Nim>. Dostęp: 16.05.2026.
- [13] Wikipedia. Nim – wolna encyklopedia. <https://pl.wikipedia.org/wiki/Nim>. Dostęp: 16.05.2026.

Spis rysunków

Załącznik nr 2 do Zarządzenia nr 228/2021 Rektora Uniwersytetu Rzeszowskiego z dnia 1 grudnia 2021 roku w sprawie ustalenia procedury antyplagiatowej w Uniwersytecie Rzeszowskim

OŚWIADCZENIE STUDENTA O SAMODZIELNOŚCI PRACY

Konrad Fugiel (134909) Oliwier Hędrzak (134913)
.....
Filip Lisowski (134942)
Imię (imiona) i nazwisko studenta

Wydział Nauk Ścisłych i Technicznych

..... Informatyka

Nazwa kierunku

.....
Numer albumu

1. Oświadczam, że moja praca projektowa pt.: Implementacja algorytmów sztucznej inteligencji na przykładzie gier Nim i Reversi – dokumentacja projektowa
 - 1) została przygotowana przeze mnie samodzielnie*,
 - 2) nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (t.j. Dz.U. z 2021 r., poz. 1062) oraz dóbr osobistych chronionych prawem cywilnym,
 - 3) nie zawiera danych i informacji, które uzyskałem/am w sposób niedozwolony,
 - 4) nie była podstawą otrzymania oceny z innego przedmiotu na uczelni wyższej ani mnie, ani innej osobie.
2. Jednocześnie wyrażam zgodę/nie wyrażam zgody** na udostępnienie mojej pracy projektowej do celów naukowo-badawczych z poszanowaniem przepisów ustawy o prawie autorskim i prawach pokrewnych.

(miejscowość, data)

(czytelny podpis studenta)

* Uwzględniając merytoryczny wkład prowadzącego przedmiot

** – niepotrzebne skreślić